

Distributed Systems

By

Dr Vidya Raj C
Professor, CSE and Dean AA

Textbook:

George Coulouris, Jean Dollimore, Tim Kindberg, Gordon Blair:
Distributed Systems – Concepts and Design, Fifth Edition, Pearson
Publications, 2012.

Module – 3

Distributed Objects and Components

Distributed File systems

<i>Objects</i>	<i>Distributed objects</i>	<i>Description of distributed object</i>
Object references	Remote object references	Globally unique reference for a distributed object; may be passed as a parameter.
Interfaces	Remote interfaces	Provides an abstract specification of the methods that can be invoked on the remote object; specified using an interface definition language (IDL).
Actions	Distributed actions	Initiated by a method invocation, potentially resulting in invocation chains; remote invocations use RMI.
Exceptions	Distributed exceptions	Additional exceptions generated from the distributed nature of the system, including message loss or process failure.
Garbage collection	Distributed garbage collection	Extended scheme to ensure that an object will continue to exist if at least one object reference or remote object reference exists for that object, otherwise, it should be removed. Requires a distributed garbage collection algorithm.

Styles of Middleware: Distributed Objects and components

- **Middleware layer** –provides higher-level of programming abstraction for development of DS
- **Distributed Object**
 - A BLOB of intelligence that live anywhere on a network
 - These objects are packed as independent pieces of code that are accessed by remote client using method invocation

Some examples:

Component standards for desktops: JavaBeans, ActiveX

Component standards for Server side: EJB, CORBABEANS, COM+

Benefits of Distributed Objects

- Interoperability – using ORBs
- Component assembly – Plus and Play with different components
- Portability – across platforms
- Co-Existence – I/F with legacy applications using ORBs
- Self managing entities – can manage on their own

Distributed Object Middleware:

- The key characteristics of distributed objects is to adopt to object-oriented programming model for development of DS applications
- Mainly objects communicate using RMI alternatively using Distributed events
- Benefits
 - Encapsulation
 - Data abstraction
 - Dynamic and extensible solution

Distributed Object Middleware:

- **Additional functions:**

- Inter object communication – RMI, CORBA, Event services
- Life cycle management – creation, migration, deletion of objects
- Activation and deactivation -

Activation: process of making an object active in DS environment by providing necessary resources for it to process incoming invocations

Deactivation: rendering an object temporarily unable to process innovation

- Persistence – Maintaining the state of an object across possible cycles of activation and deactivation, system failures

Component – based middleware:

- Software that facilitates communication and connectivity between applications or components in DS
- These solutions have been developed to overcome limitations of distributed objects middleware for application developers:
 - Implicit dependencies
 - Programming complexity
 - Lack of separation of distribution concerns
 - No support for deployment

Examples of component-based solutions: EJB, Fractal

Key characteristics of Component-based middleware:

- Abstraction
- Modularity
- Loose coupling
- Reusability
- Standard interfaces

Case Study: **CORBA**

(Common Object Request Broker Architecture)

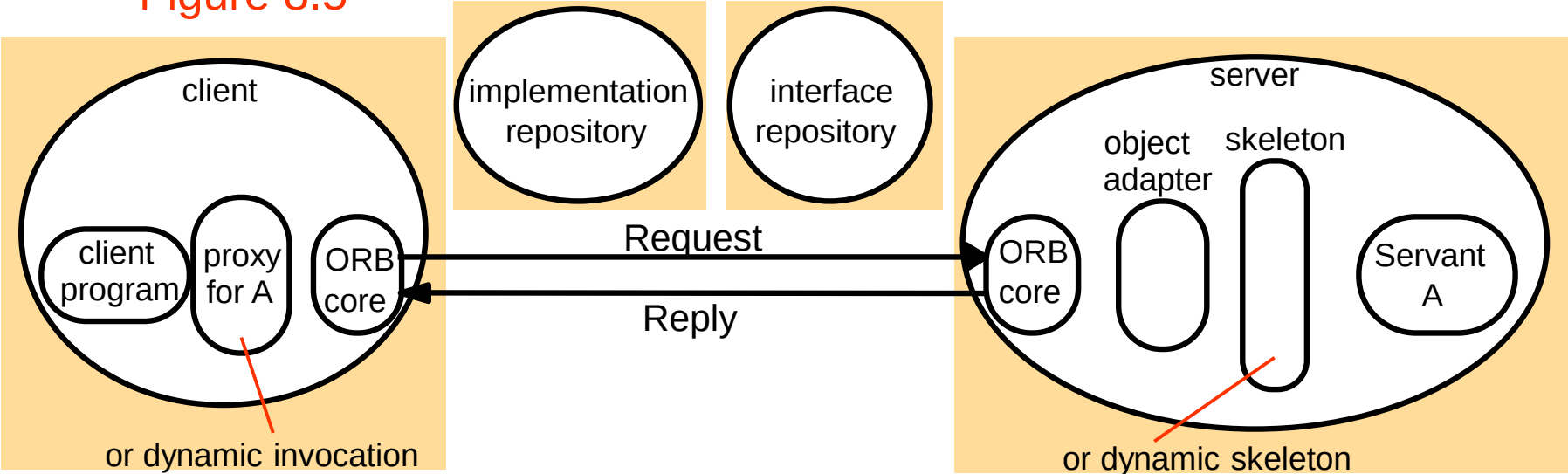
- Developed by “The Object Management Group”
- Motivation: To allow distributed objects to be implemented in any programming language and be able to communicate with another
- The OMG designed Interface Language that was independent of any specific implementation language – IDL
- CORBA’s XDR – CDR
- CORBA architecture allows for CORBA services
- A CORBA IDL interface specifies a name and a set of methods that clients can request

- **CORBA object**

- is a virtual entity
- It does not exist on its own but rather it is brought to life when, using the reference to that object, the client application requests an operation on that object
- The reference to the CORBA object is known as an Object reference
- CORBA specifies a network protocol, **IIOP – Internet Inter-ORB Protocol**, provides for transmission of ORB requests and data over TCP/IP

CORBA Architecture

Figure 8.5



The main components of the CORBA architecture

- Based on Client-server architecture, uses request-reply protocol
- Provides both static and dynamic invocation
- Components:

ORB Core:

- Role includes all functions of communication module. In addition.
- Operations enabling it to be started and stopped
- Operations to provide argument lists for requests using dynamic invocation

Object Adapter:

- To bridge the gap between CORBA objects with IDL interfaces and corresponding servant class
- This role also includes that of remote reference and dispatcher modules

The main components of the CORBA architecture

- **Object Adapter tasks:**
 - Creates remote object references for CORBA objects
 - Dispatches each RMI via a skeleton to the appropriate servant
 - Activates and deactivates servants
- An object adapter gives each CORBA object, a unique name which is part of remote object reference and it is registered in a table mapping servants
- Also each Object Adapter has its own name, specified by application programmer or generated automatically
- CORBA std for Object adapters is called as Portable Object Adapter (POA)
 - » It is called POA as it allows applications and servants to be run on ORBs produced by different developers

The main components of the CORBA architecture

- **Skeletons**

- Skeleton classes are generated in the language of server by an IDL compiler
- RMI are dispatched via the appropriate skeleton to a particular servant
- Un marshals the arguments in the request msg and marshals exceptions and results in reply msg

- **Client stubs / Proxies**

- Are in client language
- The class of proxy (for OO lang) or a set of stub procedures (for procedural langs) is generated from IDL Interface by an IDL compiler for client lang
- Marshal the arguments in invocation requests and un marshal exceptions and results in replies

The main components of the CORBA architecture

- **Implementation repository:**

- Responsible for activating registered servers on demand and for locating servers that are currently running
- The object adapter name is used to refer to the servers when registering and activating them
- It stores a mapping from names of object adapters to the path names of files containing object implementation
- Implementation repository entry:
Object adapter name Path name of implementation Hostname & port name of server

- **Interface repository**

- The role is to provide information about registered IDL interfaces to clients & servers that require it
- For an interface of given type, it can supply the names of methods and for each method, the names & types of arguments and exceptions

The main components of the CORBA architecture

- **Dynamic Invocation Interface:**

- Allows clients to make dynamic invocations on remote CORBA objects used when it is not practical to use proxies
- The client can obtain from the interface repository the necessary information about the methods available for a given CORBA object. The client may use this information to construct an invocation with suitable arguments and send it to the server.

- **Dynamic Skeletons**

- It may be necessary to add to a server a CORBA object whose interface was unknown when the server was compiled.
- If a server uses dynamic skeletons, then it can accept invocations on the interface of a CORBA object for which it has no skeleton.
- When a dynamic skeleton receives an invocation, it inspects the contents of the request to discover its target object, the method to be invoked and the arguments. It then invokes the target.

Figure 8.6

CORBA Services – a set of generic services that are useful for DS applications

<i>CORBA Service</i>	<i>Role</i>	<i>Further details</i>
<i>Naming service</i>	Supports naming in CORBA, in particular mapping names to remote object references within a given naming context (see Chapter 9).	[OMG 2004b]
<i>Trading service</i>	Whereas the Naming service allows objects to be located by name, the Trading service allows them to be located by attribute; that is, it is a directory service. The underlying database manages a mapping of service types and associated attributes onto remote object references.	[OMG 2000a , Henning and Vinoski 1999]
<i>Event service</i>	Allows objects of interest to communicate notifications to subscribers using ordinary CORBA remote method invocations (see Chapter 6 for more on event services generally).	[Farley 1998, OMG 2004c]
<i>Notification service</i>	Extends the event service with added capabilities including the ability to define filters expressing events of interest and also to define the reliability and ordering properties of the underlying event channel.	[OMG 2004d]

this figure continues on the next slide

CORBA Services (continued)

<i>Security service</i>	Supports a range of security mechanisms including authentication, access control, secure communication, auditing and nonrepudiation (see Chapter 11).	[Blakely 1999, Baker 1997, OMG 2002b]
<i>Transaction service</i>	Supports the creation of both flat and nested transactions (as defined in Chapters 16 and 17).	[OMG 2003]
<i>Concurrency control service</i>	Uses locks to apply concurrency control to the access of CORBA objects (may be used via the transaction service or as an independent service).	[OMG 2000b]
<i>Persistent state service</i>	Offers a persistent object store for CORBA, used to save and restore the state of CORBA objects (implementations are retrieved from the implementation repository).	[OMG 2002d]
<i>Lifecycle service</i>	Defines conventions for creating, deleting, copying and moving CORBA objects; for example, how to use factories to create objects.	[OMG 2002e]

Figure 8.13

Application servers

<i>Technology</i>	<i>Developed by</i>	<i>Further details</i>
<i>WebSphere Application Server</i>	IBM	[www.ibm.com]
<i>Enterprise JavaBeans</i>	SUN	[java.sun.com XII]
<i>Spring Framework</i>	SpringSource (a division of VMware)	[www.springsource.org]
<i>JBoss</i>	JBoss Community	[www.jboss.org]
<i>CORBA Component Model</i>	OMG	[Wang <i>et al.</i> 2001]
<i>JOnAS</i>	OW2 Consortium	[jonas.ow2.org]
<i>GlassFish</i>	SUN	[glassfish.dev.java.net]

Distributed file system

- A distributed file system enables programs to store and access remote files exactly as they do local ones, allowing users to access files from any computer on a network
- Goals of DFS: Network transparency and High availability
- **Distributed File Service**
 - Can be used to support the sharing of persistent storage and information
 - Enables user programs to access remote files without copying them to local disk and it provides access to files from diskless nodes
 - Name service, print service, authentication service etc

Figure 12.2

File system modules

Directory module:	relates file names to file IDs
File module:	relates file IDs to particular files
Access control module:	checks permission for operation requested
File access module:	reads or writes file data or attributes
Block module:	accesses and allocates disk blocks
Device module:	disk I/O and buffering

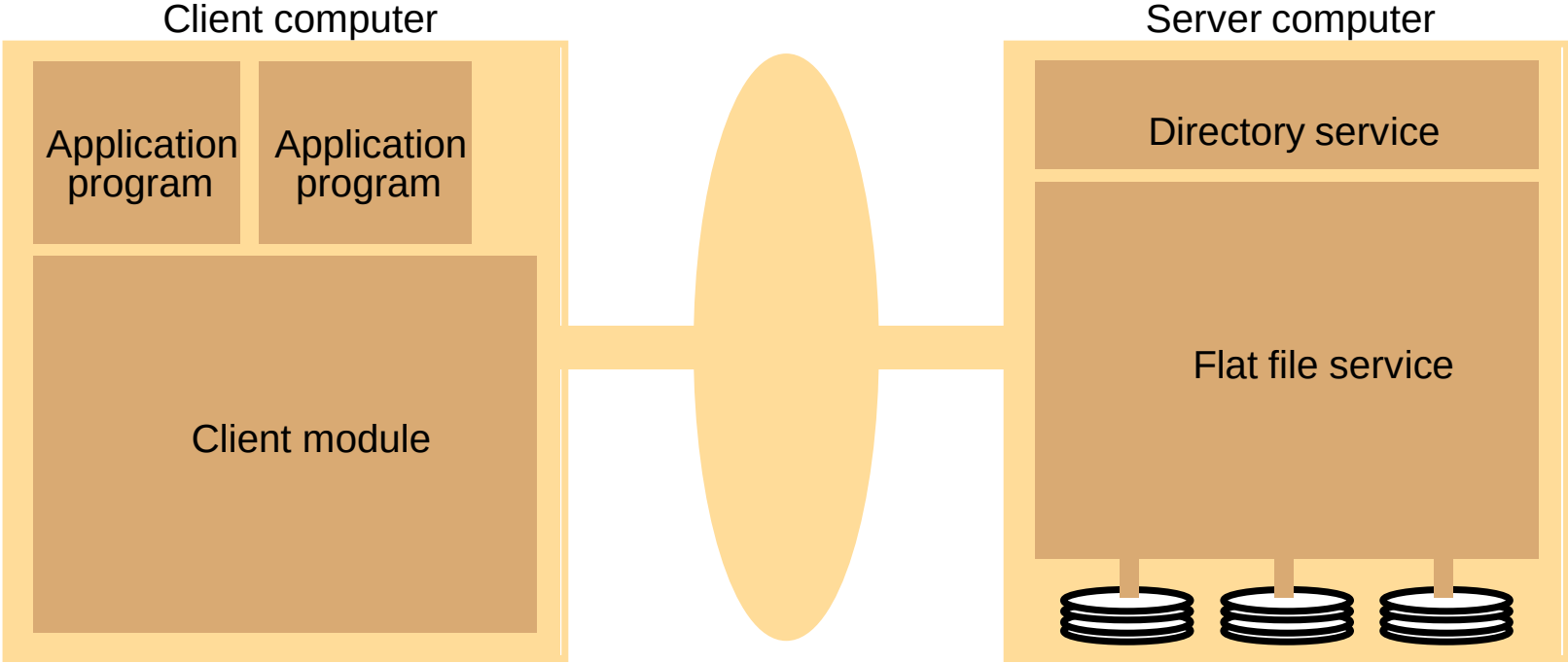
Distributed file system requirements

1. **Transparency requirements** - The file service is usually the most heavily loaded service in an intranet, so its functionality and performance are critical
 - **Access transparency:** Client programs should not be aware of distribution of files. A single set of operations is provided for access to local and remote files.
 - **Location transparency:** Client programs should see a uniform file name space. Files or groups of files may be relocated without changing their pathnames, and user programs see the same name space wherever they are executed.
 - **Mobility transparency:** Neither client programs nor system administration tables in client nodes need to be changed when files are moved.
 - **Performance transparency:** Client programs should continue to perform satisfactorily while the load on the service varies within a specified range.
 - **Scaling transparency:** The service can be expanded by incremental growth to deal with a wide range of loads and network sizes.

Distributed file system requirements

2. **Concurrent file updates** - Changes to a file by one client should not interfere with the operation of other clients simultaneously accessing or changing the same file.
3. **File Replication** - a file may be represented by several copies of its contents at different locations. This helps in multiple servers to share the load, enhancing the scalability of the service, and it enhances fault tolerance
4. **Hardware and operating system heterogeneity** • The service interfaces should be defined so that client and server software can be implemented for different operating systems and computers.
5. **Fault tolerance** – Failure due to loss of messages, temporary interruption of services in spite of failure, the system should continue its services to clients and servers
6. **Security**
7. **Efficiency**

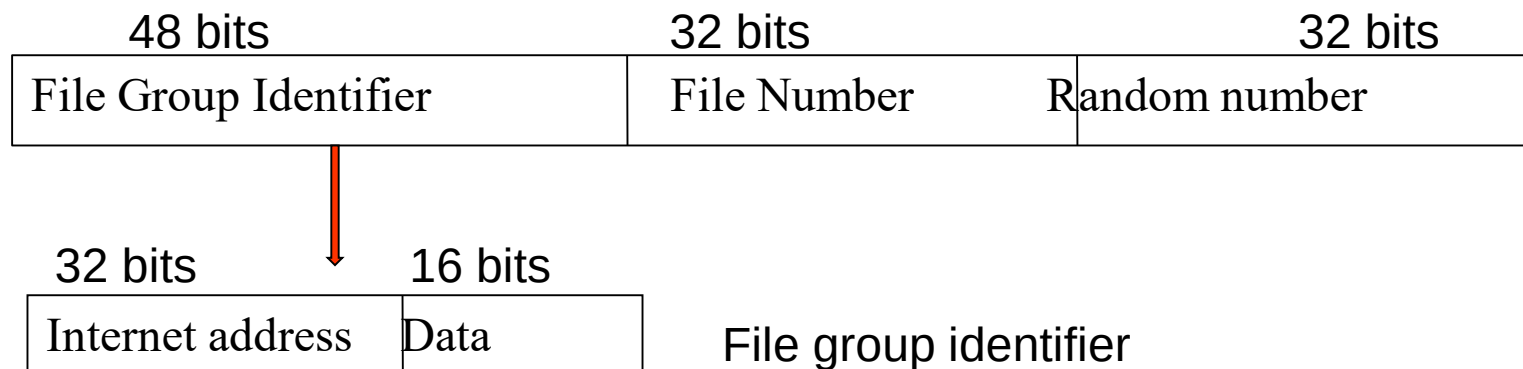
Figure 12.5
File Service Architecture



File Service Architecture – Components

- **Flat file service**

- The flat file service is concerned with implementing operations on the contents of files
- When the flat file service receives a request to create a file, it generates a new UFID for it and returns the UFID to the requester
- Unique file identifiers (UFIDs) are long sequences of bits chosen so that each file has a UFID that is unique among all of the files in a DS



- **Directory Service**

- The primary purpose of the directory service is to provide a service for translating text names to UFIDs for files
- When a new file is created, the client module must record the UFID of each file in a directory along with a text name
- This service provides the functions needed to generate and update directories and to obtain UFIDs from directories

- **Client Module**

- It is an extension of the user program
- A client module runs in each client computer, integrating and extending the operations of the flat file service and the directory service under a single application programming interface that is available to user-level programs in client computers.
- It holds information about the network locations of the flat file server and directory server process
- Uses RPC interface of flat file service and directory service

Figure 12.6

Flat file service operations

<i>Read(FileId, i, n) -> Data</i> — throws <i>BadPosition</i>	If $1 \leq i \leq \text{Length}(\text{File})$: Reads a sequence of up to n items from a file starting at item i and returns it in <i>Data</i> .
<i>Write(FileId, i, Data)</i> — throws <i>BadPosition</i>	If $1 \leq i \leq \text{Length}(\text{File}) + 1$: Writes a sequence of <i>Data</i> to a file, starting at item i , extending the file if necessary.
<i>Create() -> FileId</i>	Creates a new file of length 0 and delivers a UFID for it.
<i>Delete(FileId)</i>	Removes the file from the file store.
<i>GetAttributes(FileId) -> Attr</i>	Returns the file attributes for the file.
<i>SetAttributes(FileId, Attr)</i>	Sets the file attributes (only those attributes that are not shaded in Figure 12.3).

Figure 12.7

Directory service operations

Lookup(Dir, Name) -> FileId
— throws *NotFound*

Locates the text name in the directory and returns the relevant UFID. If *Name* is not in the directory, throws an exception.

AddName(Dir, Name, FileId)
— throws *NameDuplicate*

If *Name* is not in the directory, adds (*Name, File*) to the directory and updates the file's attribute record.
If *Name* is already in the directory: throws an exception.

UnName(Dir, Name)
— throws *NotFound*

If *Name* is in the directory: the entry containing *Name* is removed from the directory.
If *Name* is not in the directory: throws an exception.

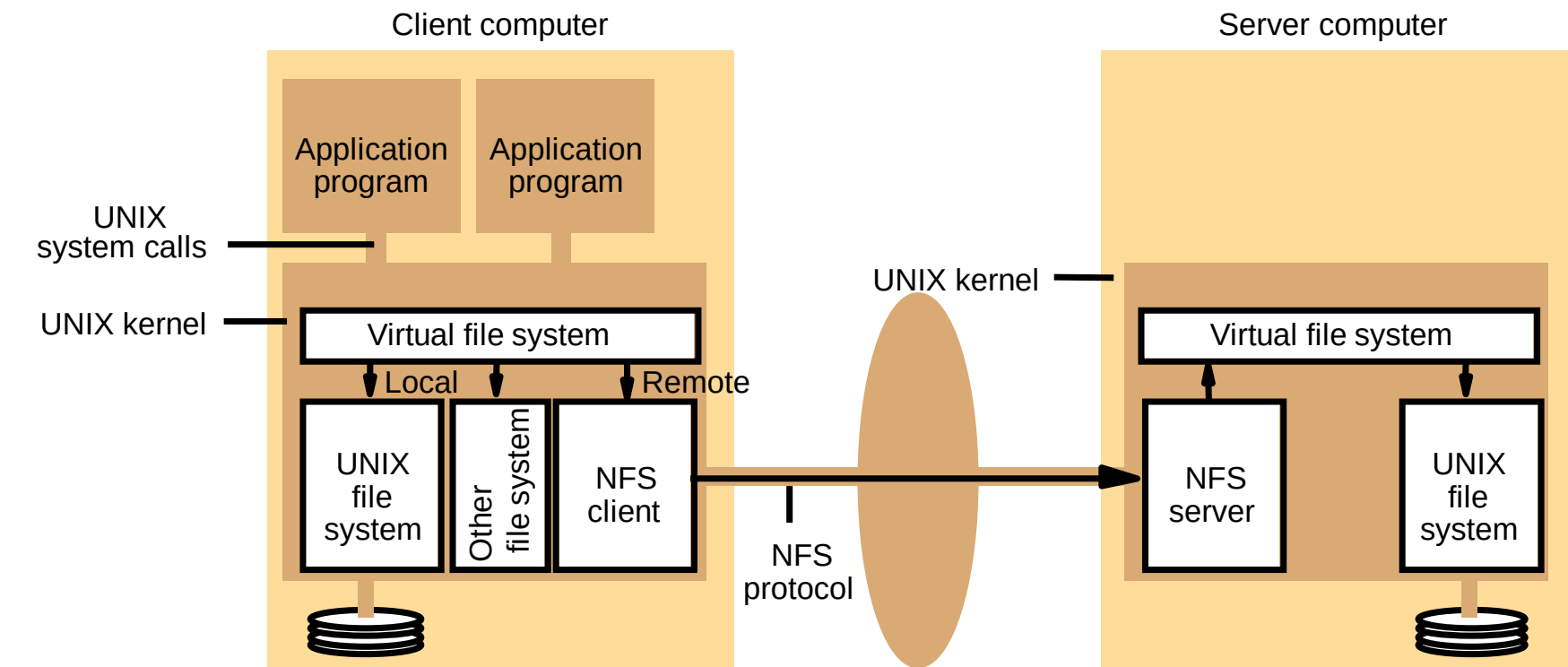
GetNames(Dir, Pattern) -> NameSeq

Returns all the text names in the directory that match the regular expression *Pattern*.

Case Study: SUN Network File system

- An important goal of NFS is to achieve a high-level support for hardware and OS heterogeneity
- All implementations of NFS support the NFS protocol
 - a set of RPCs that provide the clients to perform operations on a remote file store
 - The NFS protocol is OS-independent but was originally developed for Unix systems.
- Every computer has *NFS Client* and *NFS Server* module installed in the system kernel
- Application program or user-level client process handles operations / system calls like READ, WRITE, CLOSE

Figure 12.8
NFS architecture



- NFS Client module resides in kernel on each computer
- The NFS server module resides in the kernel on each computer that acts as an NFS server.
- When a client program request for a file in the remote file system, the request is translated into NFS protocol operations by the NFS client module and then passed to the NFS server module which holds / fetches the requested file from the file system
- The NFS client and server modules communicate using RPC

Virtual file system (VFS) module

- A VFS module, is in UNIX kernel, to distinguish between local and remote files and to translate between the UNIX-independent file identifiers used by NFS and the internal file identifiers normally used in UNIX and other file systems
- File handle - file identifiers used in NFS, in UNIX, File handle is derived using three fields
- File handle format: **FS Identifier** **i-node number** **i-node gen num**

FS Identifier – a unique num allocated to each FS when it is created

i-node number – a num, helps to identify and locator the file in a FS where the file is stored

i-node gen num – a num used to indicate the number of times an i-node number is reused

Implementation

1. The user-level client process, processes using NFS. It handles the system calls. Once the system call is parsed and parameters checked, the appropriate file handle is passes to VFS
2. VFS module that resides in kernel provides a layer of abstraction that hides the complexity of different FS implementation from users, so that development of FS becomes easier
3. VFS translates file handle to internal file identifiers used in unix or other FS
4. VFS maintains a table with one entry called v-node (or i-node in Unix) for each open file. So, using v-node / i-node it is possible to tell whether the file is local or remote.

